

6/10/2026

TryHackMe - Forensic Imaging

What is Forensic Imaging :-

Imaging = creating an exact, bit-by-bit copy of a storage device.

U capture EVERYTHING including deleted files, hidden files, unallocated space.

Why?? Because u never touch the original evidence. U work on the copy.j

Write-Blocker :-

It is a tool (hardware or software).

One-way - ur computer can read data from the suspect's drive but cannot write anything to it.

Without write blocker, just plugging the drive in will modify timestamps and taint evidence.

Audit Trail :-

Audit trail = a log of every action u performed during investigation.

Needed for court-proves what u did, in what order, at what time.

Bash commands to enable proper history logging :-

```
set -o history
shopt -s histappend
export HISTFILE=~/.bash_history
export HISTFILESIZE=-1
export HISTSIZE=-1
export HISTTIMEFORMAT="%F-%R "
```

Also good to use tools like script or ttyrec to record full terminal sessions.

Accessing the File System (Finding the Disk) :-

Before imaging, u need to identify which disk u want to image.

Step 1 - check currently mounted disks :-

```
df
```

Note :- The target disk won't appear here if it's not mounted yet. That's actually what we want (we don't mount evidence).

Step 2 - list ALL block devices (mounted or not) :-

```
lsblk -a
```

Look for the loop device that matches ur expected disk size.

For Ex :- In this room target is 1.1 GB. So u look for whichever loop device shows 1.1G. In example output it's loop11.

Step 3 - get info about the loop device :-

```
sudo losetup -l /dev/loop11
```

BACK-FILE column tells u which actual file is backing this fake disk.

Backing means :- The loop device is just a costume. The real data is in some file like /home/ubuntu/example1.img. The loop device pretends to be a disk but the data lives in that file.

Step 4 - get UUID and filesystem type :-

```
sudo blkid /dev/loop11
```

UUID = unique ID for the filesystem. Log this in ur notes.

TYPE = filesystem (ext4, NTFS, FAT32 etc.).

Note :- For physical disks u'd also use hdparm to get manufacturer, serial number, etc.

Creating the Forensic Image :-

Now u've identified the disk. Time to actually copy it.

Tools for Creating Images :-

1.dd - Standard Unix tool. Old but works.

2.dc3dd - Enhanced version of dd. Adds hashing and logging. (We'll use this one)

We'll use these two mostly but there more as follows:-

ddrescue, FTK Imager, Guymager , EWF tools (ewfacquire)

The dc3dd Command :-

```
sudo dc3dd if=/dev/loop11 of=example1.img log=imaging_loop11.txt
```

Breaking it down :-

if= - input file (where to copy FROM). Here it's the disk /dev/loop11.

of= - output file (where to copy TO). Here it's a new file called example1.img.

log= - save a report of everything that happened. Saves to imaging_loop11.txt for ur audit trail.

Output u'll see :-

- Device size in sectors and bytes
- Progress bar showing % copied
- Number of bad sectors (0 is good)
- Start and end timestamps

After it finishes, check file size :-

```
ls -alh example1.img
```

Size should match the original disk size. If yes, basic check passed.

Integrity Check :-

Same file size is not enough proof. Two different files can have same size.

To really prove the copy is identical to original we use hashing.

What is Hashing :-

Hash = a unique fingerprint of a file. Run any file through MD5 / SHA-1 / SHA-256 and u get a fixed-length string.

Rules :- Same input always gives same hash. Different input gives totally different hash. Cannot be reversed.

If even ONE byte changes between two files, the hashes will be completely different.

Hashing Both :-

```
sudo md5sum example1.img
```

```
sudo md5sum /dev/loop11
```

Output for both will be a 32-character hex string. They should be IDENTICAL.

If they match :- Copy is mathematically proven identical to original. Court-admissible.

If they don't match :- Something corrupted during imaging. Throw away and redo.

Note :- dc3dd can also hash WHILE imaging in real-time. Useful in big captures.

Other Types of Imaging + Mounting :-

Other Types of Imaging :-

1.Remote Imaging - Image a disk over the network. No physical access needed.

2.USB Images - Same idea but source is a USB drive.

3.Docker Images - Snapshot of a Docker container's filesystem and config.

Mounting the Image :-

U have the .img file now. But u can't browse files inside it directly.

Mounting = connecting the image's contents to a folder so u can navigate it.

Analogy :- Like opening a ZIP file as a folder without extracting. The .img file is sealed shut.

Mounting opens the door to the contents.

Step 1 - create an empty folder to mount into :-

```
sudo mkdir -p /mnt/example1
```

Step 2 - mount the image into that folder :-

```
sudo mount -o loop example1.img /mnt/example1
```

-o loop - tells mount to treat the file as a disk. Without this it would refuse.

Step 3 - browse the contents :-

```
ls /mnt/example1/
```

Now u can see all the files and folders that were inside the image.

When done, unmount cleanly :-

```
sudo umount /mnt/example1
```

Note :- It's umount not unmount.

Practical Exercise :-

Step 1 - find the 1GB loop device :-

```
sudo lsblk -a
```

Step 2 - create the image :-

```
sudo dc3dd if=/dev/loopXX of=exercise.img log=img.log
```

Replace loopXX with the actual one showing 1G.

Step 3 - get MD5 hash (this is ur Q1 answer) :-

```
sudo md5sum exercise.img
```

Step 4 - mount the image :-

```
sudo mkdir -p /mnt/exercise
```

```
sudo mount -o loop exercise.img /mnt/exercise
```

Step 5 - find and read flag.txt (this is ur Q2 answer) :-

```
ls /mnt/exercise
```

```
cat /mnt/exercise/flag.txt
```

Yep that's ur flag.